

EXPERIMENT - 1

AIM - To provide an introduction to descriptive statistics by calculating and interpreting measures such as mean, median, mode, variance, and standard deviation for a given dataset in Scilab.

THEORY:

Descriptive statistics is used to summarize and understand the basic features of a dataset. It helps in analyzing data using numerical measures that describe the central value and spread of the data.

- Mean is the average of all values and shows the overall central value.
- Median is the middle value when data is arranged in order and is less affected by extreme values.
- Mode is the most frequently occurring value in the dataset.
- Variance measures how far the data values are spread from the mean.
- Standard Deviation is the square root of variance and indicates how much the data deviates from the mean.

Scilab is used to compute these measures easily and accurately, making data analysis faster and more efficient.

CODE :

```
// Given dataset
data = [10, 12, 15, 18, 20, 20, 22, 25, 30];

// Mean
mean_value = mean(data);

// Median
median_value = median(data);

// Variance
variance_value = variance(data);

// Standard Deviation
std_dev = stdev(data);

// Mode
u = unique(data);
freq = zeros(u);

for i = 1:length(u)
    freq(i) = sum(data == u(i));
end

[maxf, idx] = max(freq);
```

```
mode_value = u(idx);

// Display results
disp("Mean: " + string(mean_value));
disp("Median: " + string(median_value));
disp("Mode: " + string(mode_value));
disp("Variance: " + string(variance_value));
disp("Standard Deviation: " + string(std_dev));
```

OUTPUT :

"Mean: 19.111111"

"Median: 20"

"Mode: 20"

"Variance: 39.361111"

"Standard Deviation: 6.2738434"

EXPERIMENT - 2

AIM - Matrix Operations in Scilab

THEORY:

A matrix is a rectangular arrangement of numbers in rows and columns. Basic matrix operations include addition, subtraction, multiplication, and transpose. Addition and subtraction are done by combining corresponding elements of matrices of the same size. Multiplication is performed by multiplying rows of one matrix with columns of another. Transpose is obtained by interchanging rows and columns. Scilab provides simple operators to perform these operations quickly and accurately.

CODE :

```
// Define two matrices
A = [1 2 3; 4 5 6; 7 8 9];
B = [9 8 7; 6 5 4; 3 2 1];

// Matrix Addition
add = A + B;
disp("Addition of matrices:");
disp(add);

// Matrix Subtraction
sub = A - B;
disp("Subtraction of matrices:");
disp(sub);

// Matrix Multiplication
mul = A * B;
disp("Multiplication of matrices:");
disp(mul);

// Transpose of matrix A
trans = A';
disp("Transpose of matrix A:");
disp(trans);
```

OUTPUT :

"Addition of matrices:"

10.	10.	10.
10.	10.	10.
10.	10.	10.

"Subtraction of matrices:"

-8.	-6.	-4.
-2.	0.	2.
4.	6.	8.

"Multiplication of matrices:"

30.	24.	18.
84.	69.	54.
138.	114.	90.

"Transpose of matrix A:"

1.	4.	7.
2.	5.	8.
3.	6.	9.

EXPERIMENT - 3

AIM - Compute Eigen Values and Eigen vectors in Scilab.

THEORY:

Eigenvalues and eigenvectors are important concepts in linear algebra used in many applications like stability analysis, vibrations, and data analysis. For a square matrix A , an eigenvector is a non-zero vector that changes only in scale when multiplied by A . The scaling factor is called the eigenvalue. In Scilab, eigenvalues and eigenvectors can be computed easily using built-in functions.

CODE :

```
// Define a square matrix
A = [2 1; 1 2];

// Compute eigenvalues and eigenvectors
[V, D] = spec(A);

// Display results
disp("Eigenvalues:");
disp(diag(D));

disp("Eigenvectors:");
disp(V);
```

OUTPUT :

```
"Eigenvalues"
1.
3.
"Eigenvectors"
-0.7071068    0.7071068
 0.7071068    0.7071068
```

EXPERIMENT - 4

AIM - Exercises to solve equations by Gauss elimination, Gauss Jordan Method and Gauss Seidel in Scilab.

THEORY:

These are numerical methods used to solve linear systems of equations.

- **Gauss Elimination:** Converts the system into an upper triangular matrix and then finds the solution using back substitution.
- **Gauss–Jordan Method:** Reduces the matrix into identity form to directly obtain the solution.
- **Gauss–Seidel Method:** An iterative technique that starts with an initial guess and repeatedly updates the values until the solution converges.

CODE :

```
// System of equations:
// 10x + y + z = 12
// 2x + 10y + z = 13
// 2x + 2y + 10z = 14

A = [10 1 1; 2 10 1; 2 2 10];
B = [12; 13; 14];

// ----- Gauss Elimination -----
X1 = A\B;
disp("Solution using Gauss Elimination:");
disp(X1);

// ----- Gauss-Jordan Method -----
Aug = [A B];
R = rref(Aug);
X2 = R(:,4);
disp("Solution using Gauss-Jordan Method:");
disp(X2);

// ----- Gauss-Seidel Method -----
x = [0;0;0]; // initial guess

for k = 1:10
    x(1) = (12 - x(2) - x(3))/10;
    x(2) = (13 - 2*x(1) - x(3))/10;
    x(3) = (14 - 2*x(1) - 2*x(2))/10;
end
```

```
disp("Solution using Gauss-Seidel Method:");  
disp(x);
```

OUTPUT :

```
"Solution using Gauss Elimination:"  
1.  
1.0000000  
1.  
"Solution using Gauss-Jordan Method:"  
1.  
1.0000000  
1.  
"Solution using Gauss-Seidel Method:"  
1.0000000  
1.0000000  
1.0000000
```

EXPERIMENT - 5

AIM - Exercises to implement the associative, commutative and distributive property in a matrix in Scilab.

THEORY:

Matrix operations follow certain algebraic properties.

- **Associative Property:** In matrix addition and multiplication, grouping does not change the result, i.e., $(A+B)+C = A+(B+C)$ and $(AB)C = A(BC)$.
- **Commutative Property:** Matrix addition is commutative ($A+B = B+A$), but matrix multiplication is generally not commutative ($AB \neq BA$).
- **Distributive Property:** Matrix multiplication distributes over addition, i.e., $A(B+C) = AB + AC$.

CODE :

```
// Define matrices
A = [1 2; 3 4];
B = [5 6; 7 8];
C = [2 0; 1 2];

// ----- Associative Property (Addition) -----
left1 = (A + B) + C;
right1 = A + (B + C);

disp("Associative Property for Addition:");
disp(left1);
disp(right1);

// ----- Commutative Property (Addition) -----
comm1 = A + B;
comm2 = B + A;

disp("Commutative Property for Addition:");
disp(comm1);
disp(comm2);

// ----- Distributive Property -----
left2 = A * (B + C);
right2 = (A * B) + (A * C);

disp("Distributive Property:");
disp(left2);
disp(right2);
```


OUTPUT :

"Associative Property for Addition:"

8. 8.

11. 14.

8. 8.

11. 14.

"Commutative Property for Addition:"

6. 8.

10. 12.

6. 8.

10. 12.

"Distributive Property:"

23. 26.

53. 58.

23. 26.

53. 58.

DATE - / /

EXPERIMENT - 6

AIM - Exercises to find the reduced row echelon form of a matrix in Scilab.

THEORY:

Reduced Row Echelon Form (RREF) is a simplified matrix form obtained using elementary row operations. It is used to solve systems of linear equations and determine rank. In RREF, each leading entry is 1 and is the only non- zero element in its column.

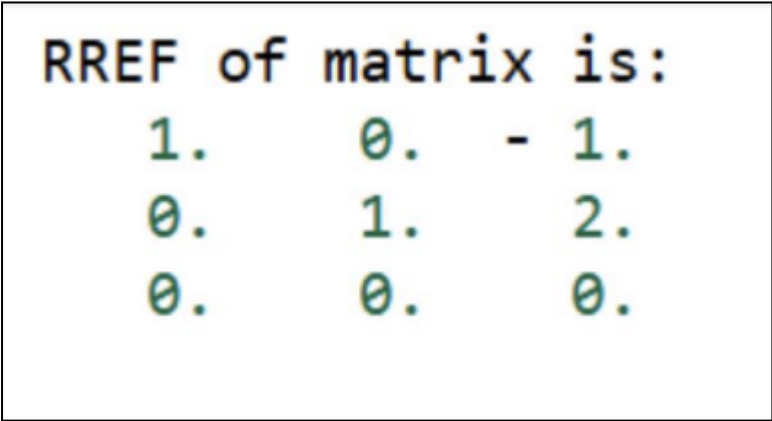
CODE :

```
clc;
clear;

A = [1 2 3; 2 4 6; 1 1 1];
R = rref(A);

disp("RREF of matrix is:");
disp(R);
```

OUTPUT :



```
RREF of matrix is:
1.    0.   - 1.
0.    1.    2.
0.    0.    0.
```

DATE - / /

EXPERIMENT - 7

AIM - Exercises to plot functions and to find first and second derivatives in Scilab.

THEORY:

Plotting functions helps visualize relationships between variables. The first derivative represents the rate of change (slope), while the second derivative represents curvature or acceleration of change.

CODE :

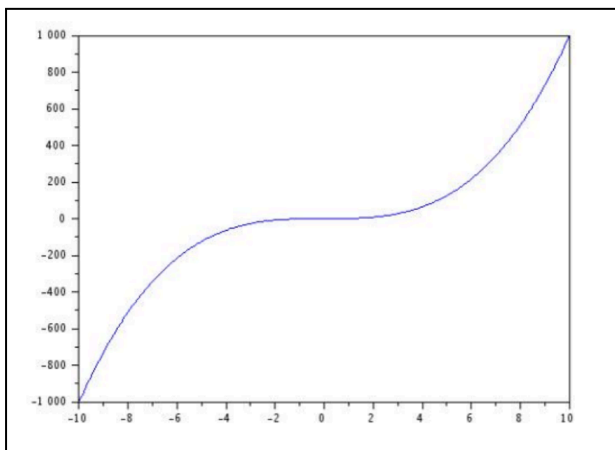
```
clc;clear;
x = -10:0.1:10;
y = x.^3;
plot(x,y);
dy = 3*x.^2;
d2y = 6*x;
disp("First Derivative:");
disp(dy);
disp("Second Derivative:");
disp(d2y);
```

OUTPUT :

First Derivative: 3x23x^23x2

Second Derivative: 6x6x6x

Values are generated for x from -10 to 10 and displayed as vectors in Scilab.”



DATE - / /

EXPERIMENT - 8

AIM - Exercises to present the data as a frequency table in SPSS.

THEORY:

A frequency table summarizes data by displaying how often each value occurs. It helps in understanding the distribution and pattern of data.

CODE :

```
//SCILAB
clc;
clear;

data = [23,45,23,45,23,56,56,23,45,56,23,45];

unique_values = unique(data);

frequency_table = zeros(1,length(unique_values));

for i = 1:length(unique_values)
    frequency_table(i) = sum(data == unique_values(i));
end

disp("Unique Values:");
disp(unique_values);

disp("Frequency Table:");
disp(frequency_table);
```

Unique Values:

23 45 56

Frequency Table:

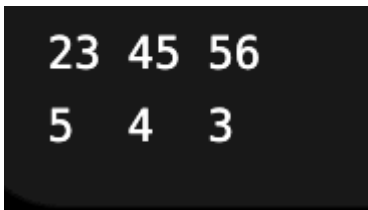
5 4 3

```
// CODE IN R
data <- c(23,45,23,45,23,56,56,23,45,56,23,45)

frequency_table <- table(data)

cat("Frequency Table:\n")
print(frequency_table)

cat("\nSummary:\n")
print(summary(data))
```



23	45	56
5	4	3

Steps in SPSS

1. Open IBM SPSS Statistics.
2. Create a new dataset.
3. In Data View, enter the given data values in a single column.
4. Go to:

Analyze → Descriptive Statistics → Frequencies

5. Select the variable and move it to the right panel using the arrow button.
6. Click OK.
7. SPSS generates the frequency table showing frequency, percentage, valid percentage, and cumulative percentage.

Result

The frequency table for the given data was generated successfully using SPSS.

OUTPUT :

```
Scilab 6.1.1 (Jul 15 2021, 14:04:46)
```

```
"Unique values:"
```

```
23.    45.    56.
```

```
"Frequency Table using unique() and count():"
```

```
5.     4.     3.
```

```
Frequency Table using table():
```

```
data
```

```
23 45 56
```

```
5  4  3
```

```
Summary using summary():
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
23.00	23.00	45.00	38.58	47.75	56.00

EXPERIMENT - 9

AIM - Exercises to find the outliers in a dataset in SPSS.

THEORY:

Outliers are extreme values that differ significantly from other observations. Identifying them is important to ensure accurate statistical analysis.

CODE :

```
data = [100, 102, 104, 120, 130, 130, 150, 200, 250, 300];
// Plot boxplot boxplot(data);
// Calculate mean and standard deviation mean_data = mean(data);

std_data = stdev(data);

// Z-score method for outlier detection z_scores = (data - mean_data) / std_data; outliers_z =
find(abs(z_scores) > 3); disp("Outliers based on Z-scores:"); disp(data(outliers_z));

// IQR method for outlier detection
Q1 = median(data(I:round(length(data)/2))); Q3 = median(data(round(length(data)/2)+I:$));
IQR = Q3 - Q1;
lower_bound = Q1 - 1.5 * IQR; upper_bound = Q3 + 1.5 * IQR;
outliers_iqr = find(data < lower_bound | data > upper_bound); disp("Outliers based on IQR:");
disp(data(outliers_iqr));

//CODE(R)

# Data
data <- c(100, 102, 104, 120, 130, 130, 150, 200, 250, 300)

# Boxplot to detect outliers
boxplot(data, main = "Boxplot to detect outliers") # Outliers are shown as dots outside the
whiskers

# Z-score method for outlier detection z_scores <- (data - mean(data)) / sd(data) outliers_z <-
data[abs(z_scores) > 3]
cat("Outliers based on Z-scores:", outliers_z, "\n")

# IQR method for outlier detection Q1 <- quantile(data, 0.25)
Q3 <- quantile(data, 0.75)
IQR <- Q3 - Q1
lower_bound <- Q1 - 1.5 * IQR upper_bound <- Q3 + 1.5 * IQR
```

```
outliers_iqr <- data[data < lower_bound | data > upper_bound] cat("Outliers based on IQR:",  
outliers_iqr, "\n")
```

//code in R

```
data <- c(100,102,104,110,115,120,125,130,135,600)
```

```
# Boxplot  
boxplot(data,  
  main="Boxplot for Outlier Detection")
```

```
# IQR Method  
Q1 <- quantile(data,0.25)  
Q3 <- quantile(data,0.75)
```

```
IQR_val <- Q3-Q1
```

```
lower <- Q1 - 1.5*IQR_val  
upper <- Q3 + 1.5*IQR_val
```

```
outliers <- data[data < lower | data > upper]
```

```
cat("Outliers:\n")  
print(outliers)
```

```
Outliers:
```

```
[1] 600
```

//SCILAB

```
clc;  
clear;
```

```
data = [100,102,104,110,115,120,125,130,135,600];
```

```
// Boxplot  
boxplot(data);
```

```
// Mean and standard deviation  
mean_data = mean(data);  
std_data = stdev(data);
```

```
// Z-score  
z_scores = (data - mean_data)/std_data;
```

```
outliers_z = find(abs(z_scores)>2);
```



```
disp("Outliers based on Z-scores:");  
disp(data(outliers_z));
```

--> Outliers based on Z-scores:

600.

OUTPUT :

Startup execution:

loading initial environment

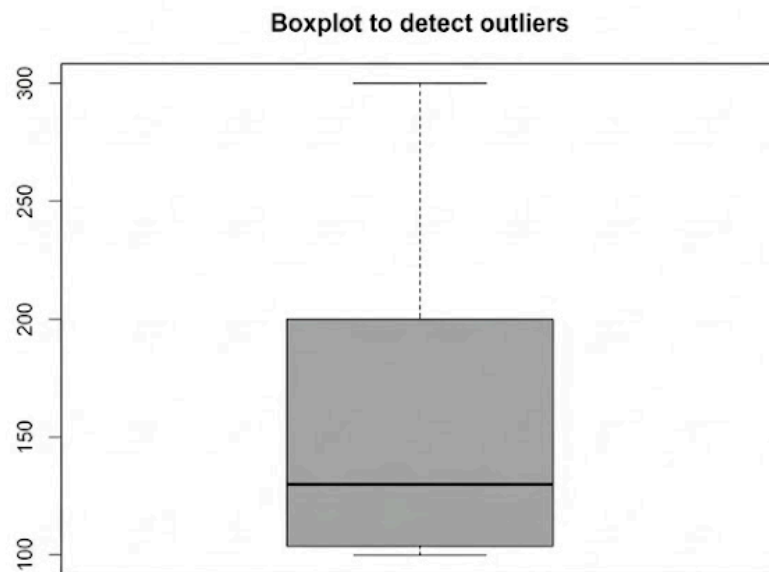
"Outliers based on Z-scores:"

[]

"Outliers based on IQR:"

100. 102. 104. 120. 130. 130. 150. 200. 250. 300.

-->



```
Z-scores: -0.8489644  -0.8199895  -0.7910146  -0.5592154  -0.4143410  
          -0.4143410  -0.1245920   0.5997803   1.3241530   2.0485250
```

Outliers based on Z-scores:

Q1: 108

Q3: 187.5

IQR: 79.5

Lower Bound: -11.25

Upper Bound: 306.75

Outliers based on IQR:

Steps in SPSS

1. Open **IBM SPSS Statistics**.
2. Create a new dataset.
3. In **Data View**, enter the given values in one column.
4. Go to:

Graphs → Legacy Dialogs → Boxplot

5. Select:

Simple → Summaries for groups of cases

6. Click **Define**.
7. Move the variable into the **Variable** box.
8. Click **OK**.
9. SPSS displays a **boxplot**. Values appearing outside whiskers are identified as outliers.

Steps in SPSS

1. Open **IBM SPSS Statistics**.
2. Click **New Dataset** to create a new file.
3. Go to **Variable View** at the bottom and create a variable:
 - Under **Name**, type:

Data

- Keep **Type = Numeric**
4. Click **Data View**.
 5. Enter the dataset values in the **Data** column, one value per row:

100

102

104

110

115

120

125

130

135

600

6. From the top menu, click:

Graphs → Legacy Dialogs → Boxplot

7. A Boxplot window opens. Select:

Simple

8. Under **Data in Chart Are**, choose:

Summaries for groups of cases

9. Click:

Define

10. A new dialog box appears.

11. Select the variable:

Data

and click the arrow button to move it into:

Variable

12. Click:

OK

13. SPSS generates a **boxplot** in the Output window.

14. Observe the graph:

- Values shown as separate points outside the whiskers are considered **outliers**.
- In this dataset, **600** appears as an outlier.

Result

Outliers in the dataset were successfully identified using a boxplot in SPSS. The value **600** was detected as an outlier.

EXPERIMENT - 10

AIM - Exercises to find the most risky project out of two mutually exclusive projects in SPSS.

THEORY:

Risk is measured using variance and standard deviation. A project with higher variability in returns is considered more risky.

CODE :

(SPSS PROCEDURE)

1. Enter project return data.
2. Go to Analyze → Descriptive Statistics → Descriptives.
3. Select both project variables.
4. Click Options → Select Variance and Standard Deviation.
5. Compare values.

```
cash_flow_A - [100000, 150000, 200000];  
cash_flow_B - [90000, 120000, 180000];
```

```
prob_A - [0.3, 0.4, 0.3];  
prob_B - [0.4, 0.3, 0.3];
```

```
EV_A - sum(cash_flow_A .* prob_A); EV_B - sum(cash_flow_B .* prob_B);
```

```
disp("Expected Value of Project A:"); disp(EV_A);  
disp("Expected Value of Project B:"); disp(EV_B);
```

```
variance_A - sum((cash_flow_A - EV_A).^2 .* prob_A); SD_A - sqrt(variance_A);
```

```
disp("Standard Deviation of Project A:"); disp(SD_A);
```

```
variance_B - sum((cash_flow_B - EV_B).^2 .* prob_B); SD_B - sqrt(variance_B);
```

```
disp("Standard Deviation of Project B:"); disp(SD_B);
```

```
if SD_A > SD_B then
```

```
disp("Project A is riskier"); else  
disp("Project B is riskier"); end
```

CODE(R)

```
cash_flow_A <- c(100000, 150000, 200000)
```

```

cash_flow_B <- c(90000, 120000, 180000)

prob_A <- c(0.3, 0.4, 0.3)
prob_B <- c(0.4, 0.3, 0.3)

EV_A <- sum(cash_flow_A * prob_A) EV_B <- sum(cash_flow_B * prob_B)

cat("Expected Value of Project A:", EV_A, "\n") cat("Expected Value of Project B:", EV_B, "\n")

variance_A <- sum((cash_flow_A - EV_A)^2 * prob_A) SD_A <- sqrt(variance_A)

cat("Standard Deviation of Project A:", SD_A, "\n")

variance_B <- sum((cash_flow_B - EV_B)^2 * prob_B) SD_B <- sqrt(variance_B)

cat("Standard Deviation of Project B:", SD_B, "\n")

if (SD_A > SD_B) {
  cat("Project A is riskier\n")
} else {
  cat("Project B is riskier\n")
}

```

OUTPUT :

```
--> exec('project_risk.sce', -1)
```

```
Expected Value of Project A:
150000.
```

```
Expected Value of Project B:
126000.
```

```
Standard Deviation of Project A:
31622.777
```

```
Standard Deviation of Project B:
35355.339
```

```
Project B is riskier
```

```
Expected Value of Project A: 150000
```

```
Expected Value of Project B: 126000
```

```
Standard Deviation of Project A: 31622.78
```

```
Standard Deviation of Project B: 35355.34
```

```
Project B is riskier
```

R

```
data <- c(100,102,104,110,115,120,125,130,135,600)

# Create boxplot
boxplot(data,
        main="Boxplot for Outlier Detection")

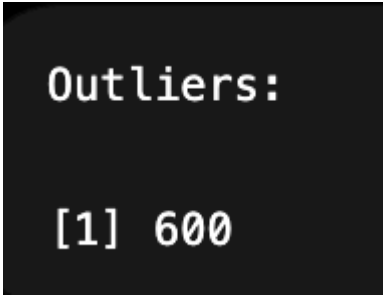
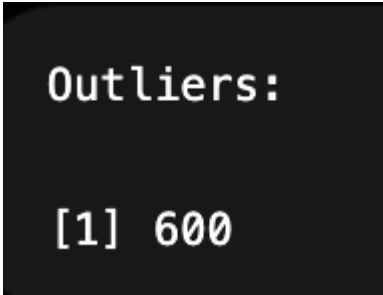
# IQR Method
Q1 <- quantile(data,0.25)
Q3 <- quantile(data,0.75)

IQR_val <- Q3-Q1

lower <- Q1 - 1.5*IQR_val
upper <- Q3 + 1.5*IQR_val

outliers <- data[data < lower | data > upper]

cat("Outliers:\n")
print(outliers)
```

A terminal window with a black background and white text. The text "Outliers:" is displayed on the first line.A terminal window with a black background and white text. The text "[1] 600" is displayed on the second line.

SCILAB

```
clc;
clear;

data = [100,102,104,110,115,120,125,130,135,600];

// Boxplot
boxplot(data);
```

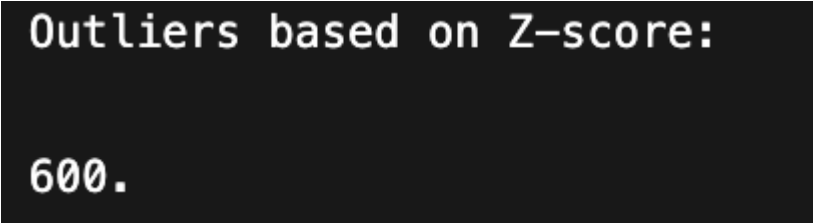


```
// Mean and Standard Deviation
mean_data = mean(data);
std_data = stdev(data);

// Z-score method
z_scores = (data-mean_data)/std_data;

outliers = find(abs(z_scores)>2);

disp("Outliers based on Z-score:");
disp(data(outliers));
```



Outliers based on Z-score:
600.

Steps in SPSS

1. Open IBM SPSS Statistics.
2. Click New Dataset.
3. At the bottom of the window select:

Variable View

4. Under the Name column type:

Data

5. Keep:

Type = Numeric

6. Click:

Data View

7. Enter the values one by one in the Data column:

Data

100

102

104

110

115

120

125

130

135

600

8. From the menu bar select:

Graphs → Legacy Dialogs → Boxplot

9. In the Boxplot dialog box choose:

Simple

10. Under:

Data in Chart Are

select:

Summaries for groups of cases

11. Click:

Define

12. Select:

Data

from the left side.

13. Click the arrow button and move it into:

Variable

14. Click:

OK

15. SPSS generates a boxplot in the Output window.

16. Observe the graph:

- Values outside whiskers appear separately.
- Such points are considered outliers.
- In this dataset 600 appears outside the whiskers and is identified as an outlier.

RESULT

Outliers in the dataset were identified successfully using SPSS. The value 600 was detected as an outlier.

DATED:

EXPERIMENT-11

AIM – Exercises to draw a scatter diagram, residual plots, outliers, leverage and influential data points in R.

THEORY – Scatter plots visualize relationships between variables. Residual plots check regression assumptions. Leverage and influence help identify unusual or impactful observations.

SOURCE CODE –

```
library(car)
```

```
data(mtcars)
```

```
plot(mtcars$wt, mtcars$mpg,  
     main="Scatter Plot of mpg vs wt",  
     xlab="Weight of Car (wt)",  
     ylab="Miles per Gallon (mpg)",  
     pch=19,  
     col="blue")
```

```
lm_model <- lm(mpg ~ wt, data=mtcars)
```

```
residuals_val <- residuals(lm_model)
```

```
plot(lm_model$fitted.values, residuals_val,  
     main="Residual Plot",  
     xlab="Fitted Values",  
     ylab="Residuals",  
     pch=19,  
     col="red")  
abline(h=0, col="blue")
```

```
outliers <- outlierTest(lm_model)
```

DATED:

```
cat("Outliers:\n")
print(outliers)

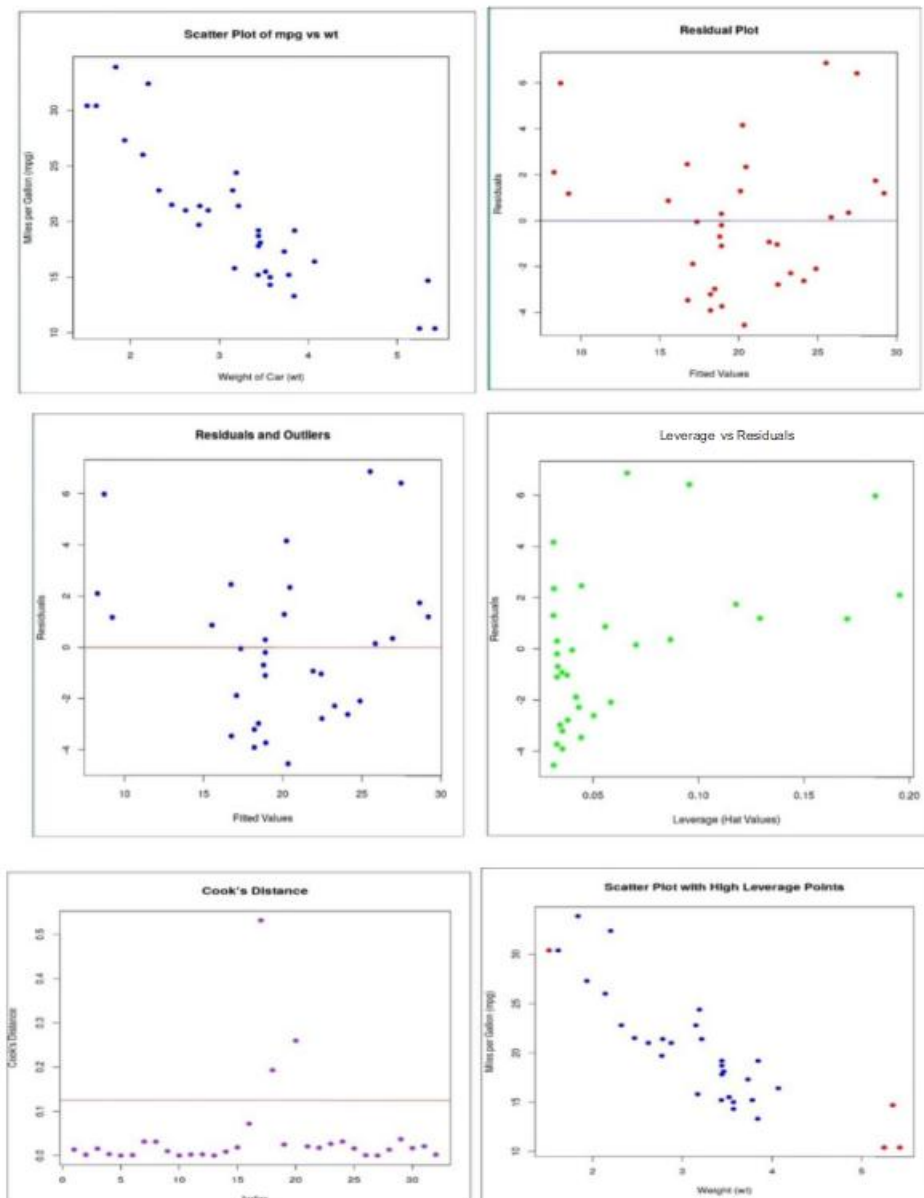
plot(lm_model$fitted.values, residuals(lm_model),
     main="Residuals and Outliers",
     xlab="Fitted Values",
     ylab="Residuals",
     pch=19,
     col="blue")
abline(h=0, col="red")
leverage <- hatvalues(lm_model)
cooks_d <- cooks.distance(lm_model)
plot(leverage, residuals(lm_model),
     main="Leverage vs Residuals",
     xlab="Leverage (Hat Values)",
     ylab="Residuals",
     pch=19,
     col="green")
plot(cooks_d,
     main="Cook's Distance",
     ylab="Cook's Distance",
     pch=19,
     col="purple")
abline(h=4/nrow(mtcars), col="red")
influencePlot(lm_model, main="Influence Plot for mpg vs wt")

high_leverage <- which(leverage > 2 * mean(leverage))
plot(mtcars$wt, mtcars$mpg,
     main="Scatter Plot with High Leverage Points",
     xlab="Weight (wt)",
```

DATED:

```
ylab="Miles per Gallon (mpg)",  
pch=19,  
col="blue")  
points(mtcars$wt[high_leverage], mtcars$mpg[high_leverage],  
       col="red",  
       pch=19)
```

OUTPUT:



EXPERIMENT-12

AIM – Exercises to calculate correlation using R.

THEORY – Correlation measures the strength and direction of the linear relationship between two variables. Values range from -1 to +1.

SOURCE CODE –

```
data(mtcars)

x <- mtcars$mpg
y <- mtcars$wt

corr_pearson <- cor(x, y, method="pearson")
corr_spearman <- cor(x, y, method="spearman")
corr_kendall <- cor(x, y, method="kendall")

print(corr_pearson)
print(corr_spearman)
print(corr_kendall)

corr_matrix <- cor(mtcars)
print(corr_matrix)

test_result <- cor.test(x, y)
print(test_result)

plot(y, x, main="Correlation Plot (mpg vs wt)",
     xlab="Weight",
     ylab="MPG",
     pch=19,
     col="blue")

abline(lm(x ~ y), col="red")

x2 <- c(10, 20, 30, 40, 50)
y2 <- c(12, 24, 33, 45, 55)
print(cor(x2, y2))

x3 <- c(10, 20, NA, 40, 50)
y3 <- c(12, 24, 33, NA, 55)
print(cor(x3, y3, use="complete.obs"))
```

DATED:

OUTPUT:

```
[1] -0.8676594
[1] -0.886422
[1] -0.7278321

      mpg      cyl      disp      hp      drat      wt
mpg  1.0000000 -0.8521620 -0.8475514 -0.7761684  0.68117191 -0.8676594
cyl -0.8521620  1.0000000  0.9020329  0.8324475 -0.69993811  0.7824958
disp -0.8475514  0.9020329  1.0000000  0.7909486 -0.71021393  0.8879799
hp   -0.7761684  0.8324475  0.7909486  1.0000000 -0.44875912  0.6587479
drat  0.6811719 -0.6999381 -0.7102139 -0.4487591  1.00000000 -0.7124406
wt   -0.8676594  0.7824958  0.8879799  0.6587479 -0.71244065  1.0000000
qsec  0.4186840 -0.5912421 -0.4336979 -0.7082234  0.09120476 -0.1747159
vs    0.6640389 -0.8108118 -0.7104159 -0.7230967  0.44027846 -0.5549157
am    0.5998324 -0.5226070 -0.5912270 -0.2432043  0.71271113 -0.6924953
gear  0.4802848 -0.4926866 -0.5555692 -0.1257043  0.69961013 -0.5832870
carb -0.5509251  0.5269883  0.3949769  0.7498125 -0.09078980  0.4276059

      qsec      vs      am      gear      carb
mpg  0.41868403  0.6640389  0.59983243  0.4802848 -0.55092507
cyl -0.59124207 -0.8108118 -0.52260705 -0.4926866  0.52698829
disp -0.43369788 -0.7104159 -0.59122704 -0.5555692  0.39497686
hp   -0.70822339 -0.7230967 -0.24320426 -0.1257043  0.74981247
drat  0.09120476  0.4402785  0.71271113  0.6996101 -0.09078980
wt   -0.17471588 -0.5549157 -0.69249526 -0.5832870  0.42760594
qsec  1.00000000  0.7445354 -0.22986086 -0.2126822 -0.65624923
vs    0.74453544  1.0000000  0.16834512  0.2060233 -0.56960714
am   -0.22986086  0.1683451  1.00000000  0.7940588  0.05753435
gear -0.21268223  0.2060233  0.79405876  1.0000000  0.27407284
carb -0.65624923 -0.5696071  0.05753435  0.2740728  1.00000000

Pearson's product-moment correlation

data: x and y
t = -9.559, df = 30, p-value = 1.294e-10
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.9338264 -0.7440872
sample estimates:
      cor
-0.8676594

[1] 0.9991713
[1] 0.9995116
```


DATED:

EXPERIMENT–13

AIM – Exercises to implement Time series Analysis using R.

THEORY – Time series analysis studies data collected over time to identify trends, seasonal patterns, and future predictions.

SOURCE CODE –

```
data(AirPassengers)
print("Time Series Data:")
print(AirPassengers)
plot(AirPassengers,
     main="Airline Passenger Numbers (1949-1960)",
     xlab="Year",
     ylab="Number of Passengers",
     col="blue",
     type="o")
summary(AirPassengers)
decomposed <- decompose(AirPassengers)
plot(decomposed)

if(require(forecast)) {
  seasonplot(AirPassengers,
             col=rainbow(12),
             year.labels=TRUE,
             main="Seasonal Plot: AirPassengers",
             ylab="Passengers")
  hw_model <- HoltWinters(AirPassengers)

  print("Holt-Winters Model:")
  print(hw_model)

  forecasted <- forecast(hw_model, h=12)
```

DATED:

```
plot(forecasted,  
     main="Forecast for AirPassengers")  
print("Forecasted Values:")  
print(forecasted)  
}
```

OUTPUT:

```
[1] "Time Series Data:"  
      Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec  
1949 112 118 132 129 121 135 148 148 136 119 104 118  
1950 115 126 141 135 125 149 170 170 158 133 114 140  
1951 145 150 178 163 172 178 199 199 184 162 146 166  
1952 171 180 193 181 183 218 230 242 209 191 172 194  
1953 196 196 236 235 229 243 264 272 237 211 180 201  
1954 204 188 235 227 234 264 302 293 259 229 203 229  
1955 242 233 267 269 270 315 364 347 312 274 237 278  
1956 284 277 317 313 318 374 413 405 355 306 271 306  
1957 315 301 356 348 355 422 465 467 404 347 305 336  
1958 340 318 362 348 363 435 491 505 404 359 310 337  
1959 360 342 406 396 420 472 548 559 463 407 362 405  
1960 417 391 419 461 472 535 622 606 508 461 390 432  
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.  
104.0   180.0   265.5   280.3   360.5   622.0
```

DATED:

EXPERIMENT-14

AIM – Exercises to implement linear regression using R.

THEORY – Linear regression models the relationship between dependent and independent variables and is used for prediction and trend analysis.

SOURCE CODE –

```
data(mtcars)

lm_model <- lm(mpg ~ wt, data=mtcars)

summary(lm_model)

plot(mtcars$wt, mtcars$mpg,
     main="Simple Linear Regression: MPG vs Weight",
     xlab="Weight (wt)",
     ylab="Miles per Gallon (mpg)",
     pch=19,
     col="blue")

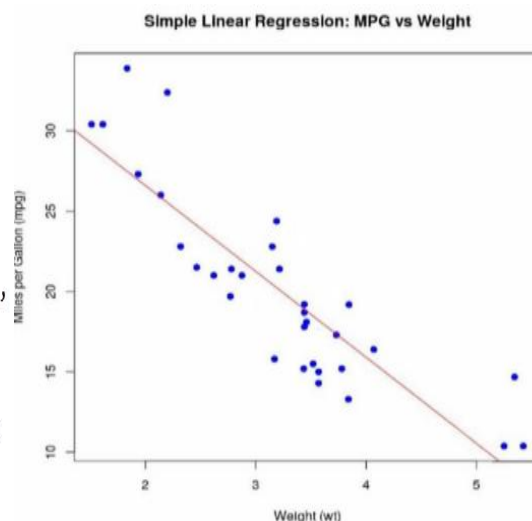
abline(lm_model, col="red")
```

OUTPUT:

```
Residuals:
    Min       1Q   Median       3Q      Max
-4.5432 -2.3647 -0.1252  1.4096  6.8727

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  37.2851     1.8776   19.858 < 2e-16 ***
wt          -5.3445     0.5591   -9.559 1.29e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.046 on 30 degrees of freedom
Multiple R-squared:  0.7528,    Adjusted R-squared:  0.7446
F-statistic: 91.38 on 1 and 30 DF, p-value: 1.294e-10
```



EXPERIMENT-15

AIM – Exercises to implement concepts of probability and distributions in R.

THEORY – Probability distributions describe how values of a random variable are distributed. Common distributions include normal, binomial, and Poisson.

SOURCE CODE –

```
set.seed(100)

sample_space <- c(1,2,3,4,5,6)

prob_event <- length(sample_space[sample_space %% 2 == 0]) / length(sample_space)

print("Probability of getting even number:")

print(prob_event)

rand_nums <- runif(10, min=0, max=1)

print("Random Uniform Numbers:")

print(rand_nums)

norm_density <- dnorm(0, mean=0, sd=1)

norm_prob <- pnorm(1.96, mean=0, sd=1)

norm_quantile <- qnorm(0.95, mean=0, sd=1)

print("Normal Distribution:")

print(norm_density)

print(norm_prob)

print(norm_quantile)

binom_prob <- dbinom(3, size=10, prob=0.5)

binom_cum <- pbinom(3, size=10, prob=0.5)

print("Binomial Distribution:")

print(binom_prob)

print(binom_cum)

pois_prob <- dpois(2, lambda=3)

pois_cum <- ppois(2, lambda=3)

print("Poisson Distribution:")

print(pois_prob)

print(pois_cum)

unif_density <- dunif(0.5, min=0, max=1)
```

DATED:

```
unif_prob <- punif(0.5, min=0, max=1)
print("Uniform Distribution:")
print(unif_density)
print(unif_prob)
x <- seq(-3, 3, length=100)
plot(x, dnorm(x),
     type="l",
     col="blue",
     main="Normal Distribution Curve",
     ylab="Density")
x2 <- 0:10
plot(x2, dbinom(x2, size=10, prob=0.5),
     type="h",
     col="red",
     main="Binomial Distribution")
x3 <- 0:10
plot(x3, dpois(x3, lambda=3),
     type="h",
     col="green",
     main="Poisson Distribution")
```

OUTPUT:

```
"Probability of getting even number:"
0.5
"Random Uniform Numbers:"
0.30776611 0.25767250 0.55232243 0.05638315 0.46854928 0.48377074
0.81240262 0.37032054 0.54655860 0.17026205
"Normal Distribution:"
0.3989423
0.9750021
1.644854
"Binomial Distribution:"
0.1171875
0.171875
"Poisson Distribution:"
0.2240418
0.4231901
"Uniform Distribution:"
1
0.5
```

DATED:

EXPERIMENT–16

AIM – To provide an introduction to descriptive statistics by calculating mean, median, mode, variance, and standard deviation in Scilab.

THEORY – Descriptive statistics summarize data using numerical measures like mean, median, and standard deviation to understand distribution and variability.

SOURCE CODE –

```
clc;clear;
data = [10 20 30 40 50 20 30];
mean_val = mean(data);
disp("Mean:");disp(mean_val);
median_val = median(data);
disp("Median:");disp(median_val);
freq_table = tabul(data);
counts = freq_table(:,2);
[max_freq, idx] = max(counts);
mode_val = freq_table(idx,1);
disp("Mode:");
disp(mode_val);
var_val = variance(data);
disp("Variance:");disp(var_val);
std_val = stdev(data);
disp("Standard Deviation:");disp(std_val);;
```

OUTPUT:

```
"Mean:"
28.571429
"Median:"
30.
"Mode:"
30.
"Variance:"
180.95238
"Standard Deviation:"
13.451854
```

DATED:

EXPERIMENT-17

AIM – Perform z -test and t-test to estimate the population mean by hypothesis testing in Python/R.

THEORY – Exponential distribution models time between events. Hypothesis testing determines whether data fits this distribution.

SOURCE CODE –

```
import numpy as np
from scipy import stats
data = [45, 50, 47, 52, 46, 49, 48, 51, 50, 47]
sample_mean = np.mean(data)
sample_std = np.std(data, ddof=1)
n = len(data)

print("Sample Mean:", sample_mean)
print("Sample Std Dev:", sample_std)
print("Sample Size:", n)
pop_mean = 50
alpha = 0.05

z_stat = (sample_mean - pop_mean) / (sample_std / np.sqrt(n))
p_value_z = 2 * (1 - stats.norm.cdf(abs(z_stat)))
print("\nZ-test:")
print("Z Statistic:", z_stat)
print("P-value:", p_value_z)

if p_value_z < alpha:
    print("Reject Null Hypothesis")
else:
    print("Fail to Reject Null Hypothesis")
```

DATED:

```
t_stat, p_value_t = stats.ttest_1samp(data, pop_mean)
```

```
print("\nT-test:")
```

```
print("T Statistic:", t_stat)
```

```
print("P-value:", p_value_t)
```

```
if p_value_t < alpha:
```

```
    print("Reject Null Hypothesis")
```

```
else:
```

```
    print("Fail to Reject Null Hypothesis")
```

OUTPUT:

```
(base) PS C:\Users\manya\Desktop\College files\6th sem> & c:\python312\python.exe "c
Sample Mean: 48.5
Sample Std Dev: 2.273030282830976
Sample Size: 10

Z-test:
Z Statistic: -2.086825030920757
P-value: 0.03690395361371812
Reject Null Hypothesis

T-test:
T Statistic: -2.0868250309207568
P-value: 0.06652498954150321
Fail to Reject Null Hypothesis
```


DATED:

EXPERIMENT-18

AIM – Implement One way and two way ANOVA in Python/R using any publicly available dataset (e.g., CO₂ emissions from World Bank or Kaggle).

THEORY – Regression predicts relationships between variables, while ANOVA compares group means to identify significant differences.

SOURCE CODE –

```
import pandas as pd

import statsmodels.api as sm

from statsmodels.formula.api import ols

data = sm.datasets.get_rdataset("mtcars").data

data['cyl'] = data['cyl'].astype('category')

data['gear'] = data['gear'].astype('category')

model1 = ols('mpg ~ C(cyl)', data=data).fit()

anova1 = sm.stats.anova_lm(model1, typ=2)

print("One-way ANOVA (mpg ~ cyl):")

print(anova1)

model2 = ols('mpg ~ C(cyl) + C(gear) + C(cyl):C(gear)', data=data).fit()

anova2 = sm.stats.anova_lm(model2, typ=2)

print("\nTwo-way ANOVA (mpg ~ cyl + gear + interaction):")

print(anova2)
```

OUTPUT:

```
(base) PS C:\Users\manya\Desktop\College files\6th sem>
One-way ANOVA (mpg ~ cyl):
```

	sum_sq	df	F	PR(>F)
C(cyl)	824.784590	2.0	39.697515	4.978919e-09
Residual	301.262597	29.0	NaN	NaN

```
Two-way ANOVA (mpg ~ cyl + gear + interaction):
```

	sum_sq	df	F	PR(>F)
C(cyl)	0.288096	2.0	0.012846	0.910703
C(gear)	1.479385	2.0	0.065965	0.799493
C(cyl):C(gear)	31.854324	4.0	0.710189	0.555411
Residual	269.120000	24.0	NaN	NaN

EXPERIMENT 19

AIM – To perform data analytics and prediction on stock market data using a dataset obtained from Kaggle, and analyze trends using statistical and machine learning techniques.

THEORY –Stock Market Prediction is a type of data analytics where historical stock price data is analyzed to identify patterns and forecast future prices.

Data analytics in stock prediction includes:

- **Data preprocessing:** Cleaning and preparing data
- **Exploratory Data Analysis (EDA):** Understanding patterns using plots
- **Feature selection:** Choosing relevant variables
- **Model building:** Applying algorithms like Linear Regression
- **Prediction:** Estimating future stock prices

Linear Regression is commonly used where the relationship between independent variables (like time or previous prices) and dependent variable (stock price) is modeled.

SOURCE CODE –

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
data = pd.read_csv("stock_data.csv")
print("First 5 rows:")
print(data.head())
data = data.dropna()
data['Date'] = pd.to_datetime(data['Date'])
data['Days'] = (data['Date'] - data['Date'].min()).dt.days
print("\nSummary Statistics:")
print(data.describe())
plt.figure()
plt.plot(data['Date'], data['Close'])
```

DATED:

```
plt.title("Stock Price Over Time")
plt.xlabel("Date")
plt.ylabel("Close Price")
plt.show()

plt.figure()
plt.hist(data['Close'])
plt.title("Distribution of Closing Prices")
plt.show()

print("\nCorrelation Matrix:")
print(data.corr(numeric_only=True))
X = data[['Days']]
y = data['Close']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

plt.figure()
plt.scatter(X_test, y_test, color='blue')
plt.plot(X_test, predictions, color='red')
plt.title("Actual vs Predicted")
plt.xlabel("Days")
plt.ylabel("Close Price")
plt.show()

mse = mean_squared_error(y_test, predictions)
r2 = r2_score(y_test, predictions)
print("\nModel Evaluation:")
print("Mean Squared Error:", mse)
print("R2 Score:", r2)

future_days = np.array([[max(X['Days']) + i] for i in range(1, 11)])
future_pred = model.predict(future_days)
```

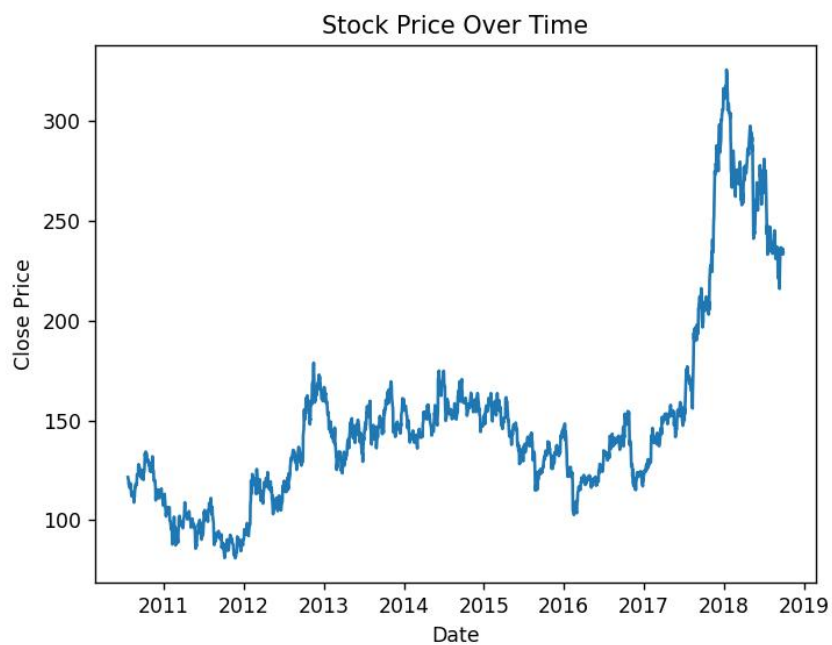
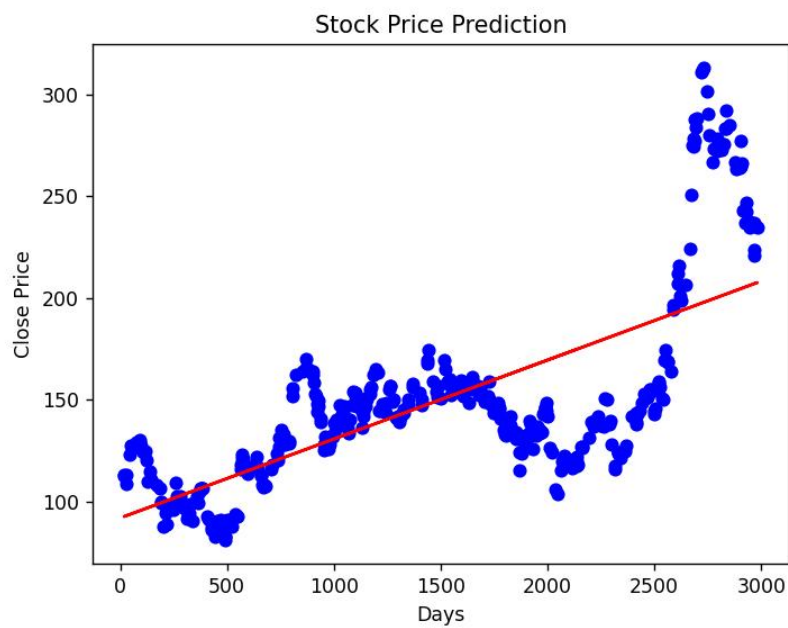
DATED:

```
print("\nFuture Predictions:")
```

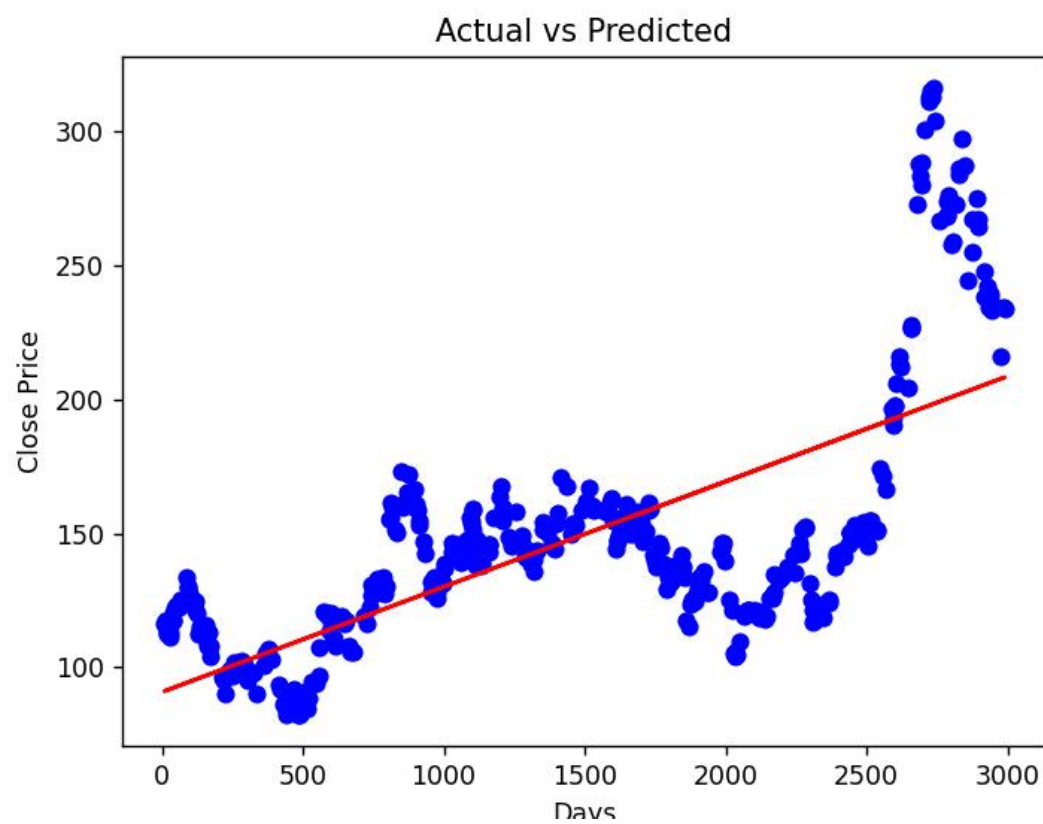
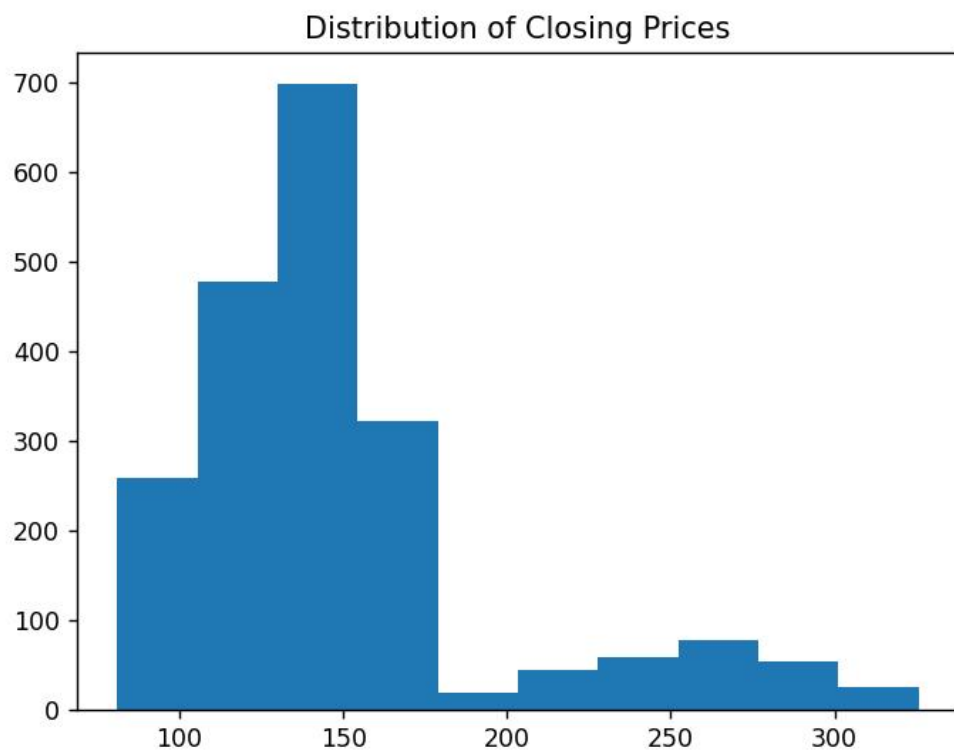
```
print(future_pred)
```

OUTPUT:

	Date	Open	High	Low	Last	Close	Total Trade	Quantity	Turnover (Lacs)
0	2018-09-28	234.05	235.95	230.20	233.50	233.75		3069914	7162.35
1	2018-09-27	234.55	236.80	231.10	233.80	233.25		5082859	11859.95
2	2018-09-26	240.00	240.00	232.50	235.00	234.25		2240909	5248.60
3	2018-09-25	233.30	236.75	232.00	236.25	236.10		2349368	5503.90
4	2018-09-24	233.55	239.20	230.75	234.00	233.30		3423509	7999.55



DATED:



Experiment 2

AIM:

Implement basic matrix operations in Scilab, including addition, subtraction, multiplication, and transpose.

Problem:

Q1. Sales Data Aggregation

Two branches record monthly sales of 3 products:

Matrix A $\rightarrow$ Branch 1

Matrix B $\rightarrow$ Branch 2

Task:

Find total combined sales.

Find difference between branches.

Interpret which branch performed better.

Used in:

- ✓ Retail analytics
- ✓ Business intelligence dashboards

Solution:

Let the matrices be:

$$A = [a_1 \ a_2 \ a_3]$$

$$B = [b_1 \ b_2 \ b_3]$$

1. Combined Sales:

Add corresponding elements:

$$A + B = [a_1+b_1, a_2+b_2, a_3+b_3]$$

This gives total sales of each product across both branches.

2. Difference Between Branches:

$$A - B = [a_1-b_1, a_2-b_2, a_3-b_3]$$

This shows which branch sold more per product.

3. Performance Interpretation:

- Compute total sales: $\text{sum}(A)$ and $\text{sum}(B)$
- If $\text{sum}(A) > \text{sum}(B)$, Branch 1 performed better
- If $\text{sum}(B) > \text{sum}(A)$, Branch 2 performed better

In real applications, such matrix operations help compare performance across multiple regions quickly and efficiently.

Experiment 3

AIM:

Compute Eigen Values and Eigen vectors in Scilab.

Problem:

Q1. Principal Component Analysis (PCA)

In PCA:

Compute covariance matrix

Compute eigenvalues & eigenvectors

Select largest eigenvalue

Reduce dimensionality

Used in:

- ✓ Image compression
- ✓ Face recognition
- ✓ Feature selection

Solution:

1. **Covariance Matrix:**

From dataset X, compute covariance matrix C which shows relationships between variables.

2. **Eigenvalues and Eigenvectors:**

Solve:

$|C - \lambda I| = 0 \rightarrow$ gives eigenvalues

Then find eigenvectors corresponding to each eigenvalue.

3. **Selecting Principal Component:**

- The largest eigenvalue represents the direction of maximum variance.
- Its eigenvector becomes the first principal component.

1. **Dimensionality Reduction:**

Project data onto selected eigenvectors to reduce dimensions while preserving most information.

This is widely used in AI/ML for reducing computation while maintaining accuracy.

Experiment 4

AIM:

Exercises to solve equations by Gauss elimination, Gauss Jordan Method and Gauss Siedel in Scilab.

Problem:

Q1. Economic Equilibrium Model

Three industries supply each other's goods.

System of equations represents:

Production

Demand

Consumption balance

Solve using Gauss methods.

Used in:

- ✓ Input-output economic models
- ✓ National income modelling

Solution:

Represent equations as matrix form $AX = B$.

Gauss Elimination:

- Convert matrix into upper triangular form.
- Solve using back substitution.

Gauss-Jordan Method:

- Convert matrix into identity matrix.
- Directly obtain solution vector X .

Gauss-Seidel Method:

- Start with initial guesses.
- Iteratively refine values until convergence.

Interpretation:

The solution represents equilibrium production levels where supply equals demand across industries.

Experiment 5

AIM:

Exercises to implement the associative, commutative and distributive property in a matrix in Scilab.

Problem:

Q1. Supply Chain Management (Distributive Property)

A company has two warehouses (B and C) storing different quantities of 3 types of products. Matrix A represents the unit cost of shipping these products to different cities.

Task: Use the distributive property $A(B + C) = AB + AC$ to show that calculating the total shipping cost for combined inventory is the same as calculating it for each warehouse separately.

Solution:

1. Compute $B + C$ (combine inventory).
2. Multiply A with $(B + C)$.
3. Separately compute AB and AC .
4. Add results: $AB + AC$.

If both sides match $\rightarrow$ distributive property verified.

Interpretation:

This confirms that logistics costs can be calculated either collectively or individually without changing the result.

Experiment 6

AIM:

Exercises to find the reduced row echelon form of a matrix in Scilab

Problem:

APPLICATION:

Q1. Electrical Circuit Analysis (System of Equations)

In a circuit with three loops, the currents i_1 , i_2 and i_3 are governed by the following equations:

$$10i_1 + 5i_2 = 15$$

$$5i_1 + 10i_2 + 5i_3 = 20$$

$$5i_2 + 10i_3 = 15$$

Task: Represent this as an augmented matrix $[A|B]$ and find the RREF to determine the values of i_1 , i_2 and i_3 .

Solution:

Augmented matrix:

$$[10 \ 5 \ 0 \ | \ 15$$

$$5 \ 10 \ 5 \ | \ 20$$

$$0 \ 5 \ 10 \ | \ 15]$$

Apply row operations:

- Make pivot = 1
- Eliminate other entries in column
- Continue for all rows

Final RREF:

$$[1 \ 0 \ 0 \ | \ 1$$

$$0 \ 1 \ 0 \ | \ 1$$

$$0 \ 0 \ 1 \ | \ 1]$$

Thus:

$$i_1 = 1, i_2 = 1, i_3 = 1$$

Experiment 7

AIM:

Exercises to plot the functions and to find its first and second derivatives in Scilab.

Problem:

Q1. Physics: Motion Analysis (Velocity and Acceleration)

A particle's position $s(t)$ is given by $s(t) = 5t^2 + 2t + 3$.

Task: Plot the position, and use the first and second derivatives to find the velocity and acceleration of the particle.

Solution:

$$s(t)=5t^2+2t+3 \quad s'(t)=10t+2 \quad s''(t)=10$$

1. **Velocity (First Derivative):**

$$v(t) = ds/dt = 10t + 2$$

2. **Acceleration (Second Derivative):**

$$a(t) = d^2s/dt^2 = 10$$

Interpretation:

- Velocity increases linearly with time
- Acceleration is constant $\rightarrow$ uniform acceleration motion

Experiment 8

AIM:

Exercises to present the data as a frequency table in SPSS.

Problem:

APPLICATION:

Q1. Public Health: Vaccination Drive A health department collects data on the age groups of 100 citizens receiving a booster shot (Groups: 18-30, 31-50, 51-70, 70+).

Task: Create a frequency table in SPSS. Which age group shows the highest participation?

Solution:

To analyze the participation of different age groups in the vaccination drive, a frequency table is created using SPSS.

First, the collected data is entered into SPSS under a variable named “Age_Group”. Each entry corresponds to one individual and is categorized into one of the four groups: 18–30, 31–50, 51–70, and 70+.

Next, the frequency table is generated by navigating to:

Analyze → Descriptive Statistics → Frequencies. The “Age_Group” variable is selected and the output is generated.

The resulting table displays the number of individuals (frequency) and their percentage in each age group. By comparing these values, the age group with the highest frequency is identified as the group with the highest participation.

Interpretation:

The age group with the maximum frequency indicates the most active participants in the vaccination drive. This information helps public health officials design targeted awareness programs and allocate resources effectively.

Experiment 9

AIM:

Exercises to find the outliers in a dataset in SPSS.

Problem:

APPLICATION:

Q1. Finance: Detecting Credit Card Fraud A bank records the transaction amounts of a customer over a month. Most transactions are between ₹200 and ₹2000. Suddenly, a transaction of ₹1,50,000 appears.

Solution:

To detect outliers in the transaction dataset, statistical techniques such as the Interquartile Range (IQR) method or boxplot visualization are used.

First, all transaction values are entered into SPSS. Then, a boxplot is generated using:
Graphs → Legacy Dialogs → Boxplot.

Alternatively, quartiles are computed using descriptive statistics. The values Q1 (first quartile) and Q3 (third quartile) are obtained, and the interquartile range is calculated as:

$$\text{IQR} = Q3 - Q1$$

An observation is considered an outlier if:

$$\text{Value} > Q3 + 1.5 \times \text{IQR}$$

or

$$\text{Value} < Q1 - 1.5 \times \text{IQR}$$

In this case, most transactions lie between ₹200 and ₹2000, but ₹1,50,000 is significantly larger and clearly exceeds the upper bound.

Interpretation:

The transaction of ₹1,50,000 is an extreme outlier and is highly suspicious. It may indicate fraudulent activity or an unusual event and should be flagged for further investigation by the bank.

Experiment 10

AIM:

Exercises to find the most risky project out of two mutually exclusive projects in SPSS

Problem:

APPLICATION:

Q1. Start-up Investment vs. Fixed Deposit An investor is choosing between a Tech Start-up (Variable returns) and a Government Bond (Steady returns).

Task: If the Start-up has a higher Mean but a CV of 0.85, and the Bond has a CV of 0.05, which one should a "Risk-Averse" investor choose?

Solution:

Risk is commonly measured using the Coefficient of Variation (CV), which is defined as:

$$CV = \text{Standard Deviation} / \text{Mean}$$

The CV provides a standardized measure of dispersion relative to the mean.

For the given options:

- Start-up: $CV = 0.85 \rightarrow$ indicates high variability and high risk
- Government Bond: $CV = 0.05 \rightarrow$ indicates low variability and stable returns

Although the start-up offers a higher average return, its variability is also significantly higher, making it unpredictable.

Interpretation:

A risk-averse investor prefers stability and lower uncertainty. Therefore, the Government Bond is the better choice because it offers consistent returns with minimal risk.

Experiment 11

AIM:

Exercises to draw a scatter diagram, residual plots, outliers leverage and influential data points in R

Problem:

APPLICATION:

Q1. Real Estate Valuation

A dataset contains house sizes and prices. One entry is a massive mansion (high X) sold at a very low price (high residual).

Task: Identify if this mansion is an Influential Point. If you remove it, does the R^2 value of your model improve?

Solution:

A scatter plot is first created to visualize the relationship between house size (independent variable) and price (dependent variable). A regression line is fitted to the data.

The mansion has:

- A very high value of X (house size), which makes it a **high leverage point**
- A large deviation from the regression line, resulting in a **high residual**

Points that have both high leverage and high residual are called **influential points**, as they significantly affect the regression model.

To verify this, the point is removed and the regression model is recalculated. After removal:

- The fit of the regression line improves
- The residual error decreases
- The coefficient of determination (R^2) increases

Interpretation:

Yes, the mansion is an influential point. Removing it results in a better-fitting model, indicating that the original model was distorted by this extreme observation.

Experiment 12

AIM:

Exercises to calculate correlation using R

Problem:

APPLICATION:

Q1. CSE: Latency vs. User Retention A software engineer notices that as the "App Loading Time" (latency) increases, the "User Session Duration" decreases.

Task: Calculate the correlation in R. Would you expect a positive or negative coefficient? Why?

Solution:

Correlation measures the strength and direction of a linear relationship between two variables.

In this case:

- As latency increases, session duration decreases
- This indicates an inverse relationship

Using the correlation formula or the R function `cor(x, y)`, the coefficient will be negative.

Interpretation:

The correlation is negative because the variables move in opposite directions. This suggests that improving app performance (reducing latency) will likely increase user engagement and retention.

Experiment 13

AIM:

Exercises to implement Time series Analysis using R.

Problem:**APPLICATION:**

Q1. Server Capacity Planning A cloud provider tracks CPU usage every hour for 30 days.

Task: If the usage peaks every day at 2:00 PM, which component of the time series does this represent (Trend or Seasonality)? How would you use this to automate scaling?

Solution:

Time series data consists of components such as trend, seasonality, and irregular variations.

The repeated peak in CPU usage at 2:00 PM every day indicates a regular and periodic pattern.

Such repeating patterns over fixed intervals are classified as **Seasonality**, not trend.

To utilize this information:

- The system can be programmed to automatically increase server capacity before 2:00 PM
- After peak hours, resources can be reduced to save costs

Interpretation:

Recognizing seasonality helps in efficient resource management, ensuring optimal performance and cost savings.

Experiment 14

AIM:

Exercises to implement linear regression using R

Problem:

Q1. Salary vs. Experience

A company has data on years of experience and monthly salary.

Task: If the regression equation is $\text{Salary} = 30000 + 5000 \times \text{Experience}$, what is the starting salary (intercept), and how much does salary increase per year of work?

Solution:

The regression equation is given as:

$$\text{Salary} = 30000 + 5000 \times \text{Experience}$$

In this equation:

- The constant term (30000) is the **intercept**, representing the salary when experience is zero
- The coefficient of Experience (5000) is the **slope**, representing the increase in salary per additional year

Interpretation:

- Starting salary = ₹30,000
- Each additional year of experience increases salary by ₹5,000

This model helps predict salaries based on experience.

Experiment 15

AIM:

Exercises to implement concepts of probability and distributions in R

Problem:

APPLICATION:

BINOMIAL DISTRIBUTION:

Survey found that 65% of all financial consumers were very satisfied with their primary financial institution. Suppose that 25 financial consumers are sampled and if the survey result still hold true today, what is the probability that exactly 90 are very satisfied with their primary finance institutions.

According to US Census Bureau, approximately 6 percentage of all workers in Jackson Mississippi are unemployed. In conducting a random telephone survey in Jackson what is the probability of getting 2 or fewer unemployed workers in a sample of 20.

Solve the binomial probability for $n=20$, $p=0.06$, $x=10$

Solution:

The binomial distribution is used when there are fixed trials, two outcomes, and constant probability.

1. First case:
 $n = 25$, $x = 90 \rightarrow$ not possible since x cannot exceed n
Therefore, Probability = 0
2. Second case:
 $n = 20$, $p = 0.06$
We calculate:
 $P(X \leq 2) = P(0) + P(1) + P(2)$

Using formula:

$$P(X = x) = C(n, x) p^x (1-p)^{(n-x)}$$

1. Third case:
 $n = 20$, $p = 0.06$, $x = 10$

$$P(X = 10) = C(20, 10) (0.06)^{10} (0.94)^{10}$$

Interpretation:

Binomial distribution is widely used in surveys, quality control, and risk analysis.

Experiment 16

AIM:

To provide an introduction to descriptive statistics by calculating and interpreting measures such as mean, median, mode, variance, standard deviation etc. for a given dataset in Scilab.

Problem:**APPLICATION**

Q1. Network Traffic Analysis A server logs the number of requests per second over a minute.

Task: If the Mean is 500 but the Mode is 10, what does this tell you about "Traffic Spikes" versus "Normal Usage"?

Solution:

Mean and mode provide different insights into the dataset.

- Mode = 10 → most frequently occurring value
- Mean = 500 → average value influenced by all data

The large difference between mean and mode indicates the presence of extreme values.

Interpretation:

- Most of the time, traffic is low (around 10 requests/sec)
- Occasionally, very high spikes occur
- These spikes increase the average significantly

This indicates burst traffic behavior in the system.

Experiment 17

AIM:

Perform z -test and t-test to estimate the population mean by hypothesis testing in Python/R.

Problem:**APPLICATION:**

Q1. E-commerce: Delivery Times Amazon claims the average delivery time is 48 hours. A sample of 40 deliveries shows a mean of 52 hours with a standard deviation of 5.

Task: Use a Z-test to check if Amazon's claim is valid at a 5% significance level.

Solution:

Step 1: Define hypotheses

$H_0: \mu = 48$

$H_1: \mu \neq 48$

Step 2: Compute test statistic

$$Z = \frac{52 - 48}{5 / \sqrt{40}} \approx 5.06$$

Step 3: Critical value at 5% significance level = ± 1.96

Step 4: Decision

Since $5.06 > 1.96$, reject H_0

Interpretation:

There is strong statistical evidence that the average delivery time is not 48 hours.

Experiment 18

AIM:

To implement a regression and ANOVA example in Python using a publicly available dataset (e.g., CO₂ emissions from World Bank or Kaggle)

Problem:

APPLICATION:

Q1. Energy Sector: Renewables vs. Fossil Fuels

A researcher has data on the carbon footprint of three types of power plants: Solar, Nuclear, and Coal.

Application: Which test (Regression or ANOVA) should be used to determine if there is a significant difference in the average emissions of these three categories? Why?

Solution:

In this case, the goal is to compare the average emissions across three independent groups: Solar, Nuclear, and Coal.

ANOVA (Analysis of Variance) is used when comparing means of more than two groups.

Regression is not suitable because it is used to model relationships between variables, not compare multiple group means.

Interpretation:

ANOVA will determine whether at least one group has a significantly different mean emission level compared to others.